Лабораторная работа 9 Оркестрация обучения и обработка отказов

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Предшествующая работа: ЛР8 «Непрерывная интеграция и quality gates»

1 Цель

Преобразовать последовательность обучения модели в наблюдаемый и параметризованный Prefect 3 flow `validate -> preprocess -> train -> evaluate`, обеспечить идемпотентные артефакты, осмысленные retries, quality gate и диагностику контролируемых отказов.

2 Результаты обучения

Каждый студент должен уметь:

- различать код обработки и код оркестрации;
- декомпозировать pipeline на tasks с явными входами и выходами;
- задавать типизированные параметры flow;
- читать состояния flow run и task run;
- применять retry только к потенциально временной ошибке;
- объяснять timeout, cache и идемпотентность;
- сохранять отчёты до возбуждения ошибки;
- связывать run с данными, параметрами, моделью и метриками;
- воспроизводить и диагностировать индивидуальный отказ.

3 Командный результат

Команда предоставляет tag `lr9-v1` и:

1. Prefect 3 flow `flight-delay-training`;
2. tasks `validate`, `preprocess`, `train`, `evaluate`;
3. CLI-параметры путей, гиперпараметров, threshold и minimum ROC AUC;
4. retry policy для стадии подготовки и отсутствие бессмысленных retries у deterministic gates;
5. каталоги минимум двух runs: успешного и контролируемого отказа;
6. JSON-отчёты каждой завершённой стадии;
7. `run-summary.json` успешного запуска;
8. тесты чистых stage-функций и отрицательного quality gate;
9. CI smoke запуска flow;
10. issue, небольшие PR, review и индивидуальные evidence cards.

4 Установка

```
python -m pip install -e ".[dev,orchestration]"
prefect version
```

На дату актуализации используется Prefect 3.8.x и диапазон зависимости `prefect>=3.8,<4`. Перед семестром преподаватель сверяет актуальную стабильную версию. Установка Prefect Cloud не обязательна: flow должен работать локально с ephemeral API или настроенным локальным server.

5 Граница обработки и оркестрации

Чистые функции `validate_stage`, `preprocess_stage`, `train_stage`, `evaluate_stage` выполняют предметную работу и тестируются без UI. Decorated tasks добавляют состояния, retries и logs. Flow определяет зависимости и общий run.

Не помещайте всю обработку внутрь одного `@flow`: тогда невозможно независимо повторить, диагностировать и оценить стадии. Не переносите расчёт метрик в YAML или deployment configuration.

6 Граф зависимостей

```
data + contract
      |
   validate
      |
  preprocess
      |
    train
      |
   evaluate
      |
run-summary.json
```

Последовательность обусловлена данными: train получает подготовленный CSV, evaluate — подготовленный CSV и model bundle. Порядок не должен зависеть от случайной очередности файловой системы.

7 Запуск успешного flow

```
python -m mlops_flight_delay.orchestration \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --run-dir reports/prefect/lr9-success \
  --model models/lr9-model.joblib \
  --n-estimators 80 \
  --max-depth 4 \
  --random-state 42 \
  --minimum-roc-auc 0.5
```

В PowerShell обратный слеш замените обратным апострофом либо выполните команду одной строкой.

Успешный каталог содержит:

- `01-validation.json`;
- `prepared.csv`;
- `03-training.json`;
- `04-evaluation.json`;
- `run-summary.json`.

8 Явные параметры

Путь к данным, контракту, каталогу run и модели не зашивается в task. Гиперпараметры и quality threshold также передаются явно.

Параметризация обеспечивает:

- повторяемость запуска;
- сравнение runs;
- тестирование граничных значений;
- будущий deployment без изменения кода;
- связь решения с evidence card.

Не включайте secrets в параметры командной строки: они попадают в историю shell и metadata run.

9 Состояния

Prefect фиксирует состояния, включая Pending, Running, Completed, Failed и Retrying. Важна причина перехода, а не цвет UI.

Если `evaluate` возбуждает исключение, task и flow получают Failed. Если `preprocess` временно не может прочитать ресурс и исчерпал retries, итог также Failed. Skipped downstream task не является корневой причиной.

Для evidence участник записывает run name/ID, параметры, первую failed task, тип исключения и восстановление.

10 Retry policy

В starter flow `preprocess` имеет две повторные попытки с небольшой задержкой. Это учебная демонстрация поведения при временном сбое ввода.

Не повторяйте автоматически:

- нарушение data contract;
- недопустимый параметр;
- недостаточную метрику модели;
- отсутствующий обязательный столбец;
- программный дефект.

Retry не исправляет детерминированную ошибку и только увеличивает время и стоимость. В зрелой версии используйте условие retry по классу исключения и экспоненциальную задержку для внешнего сервиса.

11 Контролируемый retry

```
python -m mlops_flight_delay.orchestration \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --run-dir reports/prefect/lr9-retry-student-01 \
  --model models/lr9-retry-student-01.joblib \
  --simulate-transient-failure
```

На первой попытке `preprocess` создаёт marker `.retry-once` и возбуждает `OSError`; следующая попытка продолжает работу. Для повторной демонстрации используйте новый run directory. Marker является частью учебного сценария, а не общим паттерном production-кода.

12 Детерминированный quality-gate отказ

```
python -m mlops_flight_delay.orchestration \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --run-dir reports/prefect/lr9-quality-fail \
  --model models/lr9-quality-fail.joblib \
  --minimum-roc-auc 1.1
```

`evaluate` сначала записывает `04-evaluation.json` с `passed: false`, затем возбуждает ошибку. Такой отказ не повторяется: порог 1.1 недостижим для ROC AUC и требует исправления параметра или явного решения владельца, а не retry.

13 Идемпотентность

Повтор одного запуска с теми же входами и random seed должен давать тот же подготовленный dataset hash и метрики в пределах детерминированного алгоритма. Task не должна бесконтрольно дописывать строки в общий файл.

В ЛР9 output directory задаётся явно. Для нового эксперимента создаётся новый каталог; для осознанного повторения допускается атомарная перезапись собственных файлов. Общий mutable `latest/` запрещён как единственное evidence.

14 Артефакты и происхождение

`run-summary.json` связывает:

- название и порядок стадий;
- SHA256 исходных и подготовленных данных;
- путь model bundle;
- параметры обучения;
- test metrics;
- итог quality gate.

Сам путь не является глобальным идентификатором. В следующих ЛР manifest будет дополнен MLflow run/model version и registry information.

15 Logs

Task logs содержат количество строк, hash и метрики, но не полный dataset. Исключение должно объяснять действие: какой gate не пройден и где лежит report.

Не логируйте token, `.env`, персональные данные, полный request payload и содержимое закрытого dataset. Prefect UI облегчает поиск, но политика данных остаётся ответственностью команды.

16 Timeout и cache

Timeout ограничивает зависшую работу и должен учитывать нормальное время стадии. Слишком короткий timeout создаёт ложные отказы, слишком большой задерживает реакцию.

Cache может пропустить повторное вычисление при совпавшем ключе. Он вводится только после определения состава ключа: версия кода, hash данных, параметры и relevant environment. Cache по имени файла недостаточен. В обязательном starter flow cache намеренно не включён, чтобы студент сначала доказал идемпотентность.

17 Тестирование

Обязательные tests:

1. положительная последовательность создаёт четыре stage artifacts;
2. model bundle содержит заданный random seed;
3. evaluation report имеет четыре метрики;
4. недостижимый quality threshold вызывает ValueError;
5. отрицательный report записан до ошибки;
6. retry policy preprocess равна двум попыткам;
7. CI выполняет flow smoke.

Тестирование чистых stage-функций быстро локализует дефект. Отдельный integration smoke доказывает, что decorators и flow wiring исполняются.

18 Индивидуальные failure-сценарии

Каждый участник выбирает отдельный сценарий:

Сценарий	Ожидаемая task	Ожидаемое поведение
controlled transient input	preprocess	Retrying, затем Completed
отсутствующий столбец	validate или preprocess	Failed без полезного retry
повреждённая дата	validate/preprocess	Failed с диагностикой поля
threshold ROC AUC 1.1	evaluate	report `passed=false`, Failed

Сценарий	Ожидаемая task	Ожидаемое поведение
путь model недоступен	train	Failed, downstream не выполняется
неверный тип параметра	flow validation	запуск отклонён до обработки

Сценарий не должен повреждать общий `main` или удалять чужие artifacts.

19 Командные роли

Для команды 2–5 человек роли объединяются или распределяются:

- владелец validate/data contract;
- владелец preprocess и идемпотентности;
- владелец train/parameters;
- владелец evaluate/quality gate;
- владелец orchestration tests и failure review.

Роль координатора без технического или проверочного артефакта не считается самостоятельным вкладом. Роли меняются относительно ЛР8.

20 Индивидуальная сдача участника команды

Командный результат: tag `lr9-v1`, flow, tests, успешный run и набор failure runs.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР9, commit/tag
Роль	роль в ЛР9 и отличие от ЛР8
Задача	issue с наблюдаемым критерием приёмки
Личный артефакт	task, параметр, test, report или review
Успешный run	имя/ID, параметры и итоговый summary
Отказ	личный сценарий, состояние, первая ошибка
Retry	число попыток и обоснование применимости
Происхождение	data hash, model path, parameters и metrics
Решение	выбранная граница task и альтернатива
Риск	повтор, частичный output, cache или secret
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: личная issue закрыта через PR, review выполнен другим участником, успешный run воспроизводим, failure run индивидуален, JSON evidence соответствует commit, участник объясняет retry и идемпотентность.

21 Защита

За 3–4 минуты участник:

1. показывает свою task или test в tag `lr9-v1`;
2. запускает flow с изменённым безопасным параметром;
3. открывает состояние и JSON-артефакт;
4. воспроизводит назначенный failure;
5. называет первую failed task и downstream effect;
6. объясняет, нужен ли retry;
7. связывает data hash, параметры, модель и metric;

8 показывает исправление или восстановление.

Преподаватель может изменить threshold, random seed, путь данных или попросить классифицировать новую ошибку как transient/deterministic.

22 Повторная сдача и стоп-факторы

Повторная сдача выполняется через issue, отдельный PR, новые run directory и evidence card. История первой попытки сохраняется.

Стоп-факторы:

- весь pipeline оформлен одной task;
- параметры зашиты в код и не видны в run;
- нарушение data contract повторяется retries и скрывается;
- failed report не сохранён;
- flow считается успешным после failed quality gate;
- общий mutable каталог перезаписывает evidence других участников;
- logs содержат secret или закрытые данные;
- один failure-сценарий выдан всей команде;
- студент не может связать run с commit и входами;
- скриншот UI заменяет код и reports.

23 Критерии оценивания

Критерий	Баллы	Полное выполнение
Декомпозиция flow	2	четыре tasks, явные зависимости и границы
Параметры и происхождение	2	входы, hash, параметры, model и metrics связаны
Отказы и retries	2	transient retry и deterministic failure различаются
Artifacts и идемпотентность	1	reports создаются безопасно и воспроизводимо
Тесты и CI	1	положительный, отрицательный и flow smoke
Индивидуальная защита	2	личная task/failure, диагностика и решение

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

24 Чек-лист

☐ Команда состоит из 2–5 человек, роли ротированы.

☐ Используется Prefect 3 из проверенного диапазона.

☐ Flow содержит validate, preprocess, train и evaluate.

☐ Все пути и параметры передаются явно.

☐ Validation failure останавливает downstream.

☐ Preprocess retry демонстрируется контролируемо.

☐ Deterministic quality failure не маскируется retry.

☐ Каждая стадия создаёт проверяемый report.

☐ Успешный run содержит `run-summary.json`.

☐ Tests проверяют положительный и отрицательный сценарии.

 CI выполняет Prefect flow smoke.

 Каждый участник защищает собственный failure-сценарий.

 Итоговая версия отмечена tag `lr9-v1`.

25 Связь с ЛР10

В ЛР10 успешная модель из flow будет регистрироваться в MLflow Model Registry. Quality gate станет условием присвоения alias, а release manifest свяжет flow run, MLflow run, model version и решение о rollback.

26 Официальные источники

- Prefect documentation: https://docs.prefect.io/v3/
- Prefect flows and tasks: https://docs.prefect.io/latest/tutorial/flows
- Prefect package releases: https://pypi.org/project/prefect/
- Prefect source repository: https://github.com/PrefectHQ/prefect
- Python argparse: https://docs.python.org/3/library/argparse.html